FUN GC ／ UNITY講座シリーズ

【Unity基礎】

オブジェクト指向をC#で学ぼう

第2回　カプセル化



2026年10月21日　講師：嘉屋悠月

TODAY ・ 今日のゴール

3つできるようになろう



守れる

外から直接さわれないよう、private でデータを守れる



入口を作れる

メソッドやプロパティで、決まった入口だけから変更できる



壊れない

まちがった値を入れられても、speed が壊れないクラスを作れる

POINT　用語は後から。まず動かして体験してから、名前を付けよう。

TODAY・90分

今日の流れ

01	なぜデータを守るの？	15分
02	private とメソッドで守る	20分
03	プロパティ	10分
04	Unityでのカプセル化	10分
05	演習：Mover の速さを守ろう	30分
06	まとめ・次回予告	5分

前回、こんな書き方をしたよね

第1回のコードから

```
Enemy slime = new Enemy();
slime.hp = 10;

// Spawner.cs
m.speed = Random.Range(1f, 8f);
```

外から自由に
書き換えられた。

「インスタンス.フィールド = 値」
便利だったけど……
本当にこのままで大丈夫？

今日のコードは clone と pull で読み込む

1. GitHub Desktop で「File → Clone repository」。yugetu12/fungc-unity-course を選ぶ（初回だけ。Unity プロジェクトとは別の場所に）
2. **左上の「Current Branch」で unity-base-2 に切り替える**（2回目からは「Fetch origin → Pull」）
3. clone したフォルダの scripts にある .cs を、Unity の Project ウィンドウにドラッグ＆ドロップ
4. MonoBehaviour（GameManager・Mover・Spawner）は GameObject にアタッチ。再生して動きを確かめる

⚠ 1つずつ置き換える

Enemy.cs や Mover.cs は、章ごとのフォルダに同じ名前で入っている。同時に入れず、今のファイルと置き換える。

GitHub Desktop が使えない人は、GitHub の「Code → Download ZIP」でもOK。

SECTION

01

なぜデータを守るの？



public のままだと何が起きるか、
わざと壊して確かめてみよう

public だと、ありえない値も入る

だれかが書いたコード

```
Enemy slime = new Enemy();
slime.hp = 10;

slime.hp = -999;
    // HPがマイナス！？
slime.hp = 99999;
    // 不死身のスライム
m.speed = -50f;
    // 逆走しながら爆速
```

① エラーにならない

文法としては正しいので、
コンパイラは止めてくれない。

② バグは遠くで起きる

「HPが変なのに死なない」——
原因の1行を探すのが大変。

③ チーム制作で起きやすい

決まりを知らない人が、
悪気なく書き換えてしまう。

カプセル化は「中身を隠して、入口だけ見せる」

隠す

データ（フィールド）は、
外から直接さわらせない。

入口を作る

変えたいときは、
決められたメソッドを通してもらう。

入口でチェック

「HPは0より下げない」などの
ルールを、入口の中で守る。



SECTION

02

private とメソッドで守る



フィールドを隠して、
入口になるメソッドを用意しよう

public と private のちがい

⚠ NG　だれでも書き換えられる

```
public int hp;
```

クラスの外からでも
slime.hp ＝ -999; が通ってしまう。

✓ OK　クラスの中だけ

```
private int hp;
```

外から slime.hp と書くと
その場でエラーになる。

💡 **エラーは「守れている」証拠。こわがらなくてOK。**

入口になるメソッドを作る

Enemy.cs

```
public class Enemy
{
    private int hp = 100;

    public void TakeDamage(int damage)
    {
        hp -= damage;
        if (hp < 0)
        {
            hp = 0; // 0より下げない
        }
    }
}
```

① hp は private

外から直接は書き換えられない。

② 入口は public メソッド

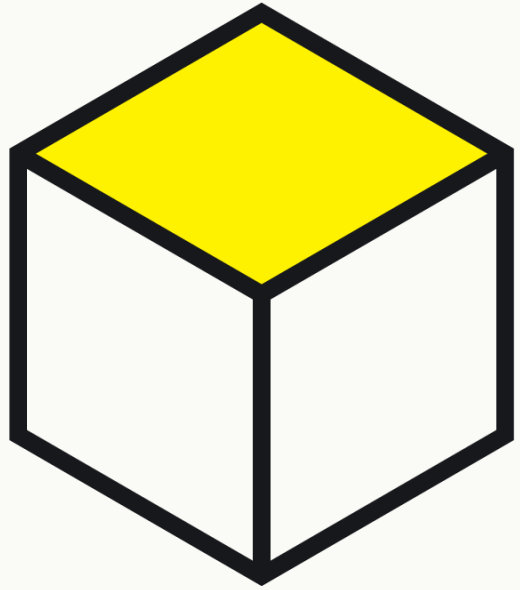
外からは slime.TakeDamage(30);
のように「お願い」する。

③ ルールは1か所に

「0より下げない」をここに書けば、
すべての敵で必ず守られる。

POINT ここだけは覚える

データは private、
さわり方はメソッドで。



迷ったら、まず private にしておこう。

SECTION

03

プロパティ



private にしたら、HPバーに表示できない？
「読めるけど書けない」を作ろう

読めるけど書けない：プロパティ

Enemy.cs

```
public class Enemy
{
    public int Hp { get; private set; } = 100;

    public void TakeDamage(int damage)
    {
        Hp -= damage;
        if (Hp < 0) Hp = 0;
    }
}
```

```
Debug.Log(slime.Hp); // OK：読める
slime.Hp = 999; // エラー：書けない
```

① get ＝ 読む

public なので、だれでも読める。

② private set ＝ 書く

書けるのはクラスの中だけ。

③ 名前は大文字から

プロパティは Hp、
フィールドは hp と書き分ける。

SECTION

04

Unityでのカプセル化



private にすると Inspector から消える？
Unity ならではの書き方をおぼえよう

Unity流：[SerializeField] private

⚠ これまで　public

```
public float speed = 5f;
```

Inspector に出る。でも、
ほかのスクリプトからも書き換えられる。

✓ これから　守った形

```
[SerializeField]
private float speed = 5f;
```

Inspector にはちゃんと出る。
ほかのスクリプトからは書き換えられない。

💡 **「Inspector で調整したい値」は、この形が Unity の定番。**

EXERCISE

演習

目安時間

30分

Mover の速さを守ろう

ゴール　Spawner が変な値を渡しても、
キューブの速さが 1〜8 の間に収まる

1. 05_exercise の Mover.cs と Spawner.cs を読み込む（speed は [SerializeField] private 済み）
2. Spawner の m.speed の行がエラーになるのを確認（成功のしるし）
3. **Mover の TODO 3：SetSpeed(float value) を作り、1〜8 に収める**
4. Spawner の TODO 4：-5〜15 のランダム値を SetSpeed に渡して確認

早く終わった人へ

Mover に読むだけの Speed プロパティを作り、出した敵の速さを Console に表示
／ Enemy に Heal() を足し、HP が 100 を超えないようにする

解答例：Mover.cs と Spawner.cs

Mover.cs

```
public class Mover : MonoBehaviour
{
    [SerializeField] private float speed = 5f;

    public void SetSpeed(float value)
    {
        if (value < 1f) value = 1f;
        if (value > 8f) value = 8f;
        speed = value;
    }

    void Update() { /* 矢印キーで動く（前回のまま） */ }
}
```

```
// Spawner.cs の m.speed = … の行を書き換え
m.SetSpeed(Random.Range(-5f, 15f));
```

① 値は隠す

Inspector では変えられるが、ほかのスクリプトからは直接さわれない。

② 入口は SetSpeed

外から速さを変えるには、必ずここを通る。

③ チェックは入口で

-5 を渡しても 1 に、15 を渡しても 8 になる。

06 ・ まとめ

今日できるようになったこと



private で隠す

データは外から直接さわらせない。
迷ったら private



入口を作る

変更はメソッド、読むだけならプロパティ。ルールは入口で守る



Unity では

Inspector に出したい値は
[SerializeField] private

NEXT

第3回　継承

Enemy を丸ごとコピーせずに、Slime や Boss を「違う部分だけ」で作れるしくみを学びます。